The Association List *(alist)* Emacs Lisp Library: An Overview

The Association List Emacs Lisp library provides a comprehensive API to work with *alists*. It has been written by Troy Pracy and the latest release is 0.6.1. This eighth article in the GNU Emacs series explores the regular and anaphoric function variants for creating, retrieving and updating *alists*. It also reviews the constructs for mapping, filtering, folding and looping of *alists*.

The source code featured in this article is available at *https://github.com/troyp/asoc.el* and released under the GNU General Public License v2.

Installation

The *asoc.el* source file can be obtained from *https://github.com/troyp/asoc.el* and added to your Emacs init configuration file so that it gets loaded on startup.

You can then use the library with the following Emacs Lisp code snippet:

```
(require 'asoc)
```

Usage

We will begin with the constructor API available in *asoc.el* for creating association lists.

Constructors

The *asoc-make* function takes a set of keys and returns an association list. If you give a default value, then all the keys are initialised with this value, as shown below:

```
(asoc-make &optional keys default) ;; Syntax

(asoc-make '(a b c d e))
((a) (b) (c) (d) (e))

(asoc-make '(a b c d e) '(0))
((a 0) (b 0) (c 0) (d 0) (e 0))
```

The *asoc-copy* function returns a shallow copy of an *alist* that is passed to it as an argument. For example:

```
(asoc-copy alist) ;; Syntax

(let ((foo (asoc-copy '((a 1) (b 2) (c 3) (d 4) (e 5)))))
  foo)
((a 1) (b 2) (c 3) (d 4) (e 5))
```

You can combine keys and values together to create an association list using the *asoc-zip* API. If there are more keys than values, then these additional keys have a nil value. A couple of examples are given below:

```
(asoc-zip keys values) ;; Syntax

(asoc-zip '(a b c) '(1 2 3))
((a . 1) (b . 2) (c . 3))

(asoc-zip '(a b c d e) '(1 2 3))
((a . 1) (b . 2) (c . 3) (d) (e))
```

If you want to return an *alist* with the duplicate keys removed, you can use the *asoc-uniq* function. In the following example, only the first value for the key *b* is returned.

```
(asoc-uniq alist) ;; Syntax

(asoc-uniq '((a 1) (b 2) (b 3) (c 4) (d 5)))
((a 1) (b 2) (c 4) (d 5))
```

The *asoc-merge* function helps you to merge multiple *alists*. If the same keys are present in multiple lists, the key in the latter *alist* takes precedence. If identical keys are present in the same